Skill-15

Name:k.vaishali

Rollno: 2520030117

Section: 8

Experiment No. 15 (Q15)

Title: Combined Redirection, Stream Merging and Complex Command Execution in a Custom Shell **Aim:**

To parse combined redirection, duplicate file descriptors, merge streams, verify ordering, handle edge cases and test integrated outputs. To parse complex commands, combine pipes and redirections, resolve execution order, validate syntax, generate execution plans and test scenarios.

Objective:

Part 1 – Combined Redirection and Stream Merging

- To parse a command line containing more than one redirection operator.
- To duplicate file descriptors so that one stream points at the same file as another.
- To merge standard error into standard output using the 2>&1 form.
- To verify that redirections are applied strictly in the order they are written.
- To handle edge cases such as a missing file name after an operator.
- To test the integrated output produced when both streams are written to the same file.

Part 2 – Complex Commands

- To parse a command line containing both pipes and redirections.
- To combine pipe connections with per-stage redirections.
- To resolve the correct execution order of pipe setup and redirection.
- To validate the syntax of the whole command line before any process is created.
- To generate an explicit execution plan describing every system call.
- To test valid and invalid scenarios, including conflicting redirections.

Theory:

1. Combined Redirection

A single command may carry several redirections at once, for example "ls a b > out.txt 2> err.txt". Each operator affects one descriptor only, so standard output and standard error can be sent to different files, to the same file, or one can be discarded while the other is kept. The command itself is unaware of any of this.

2. Duplicating File Descriptors

dup2(oldfd, newfd) closes newfd if it is open and makes it refer to the same open file description as oldfd. Both descriptors then share the same file offset, which is exactly what makes merging work: writes through either descriptor advance the same position in the file instead of overwriting each other.

```
dup2(fd, STDOUT_FILENO); /* descriptor 1 now refers to the opened file */
dup2(STDOUT_FILENO, STDERR_FILENO); /* 2>&1 : descriptor 2 copies descriptor 1 */
```

3. Merging Streams with 2>&1

The operator 2>&1 does not name a file. It means "make descriptor 2 a duplicate of whatever

descriptor 1 currently refers to". If standard output has already been redirected to a file, standard error follows it into that file and the two streams appear interleaved in one place. The reverse form 1>&2 merges standard output into standard error instead.

4. Verifying the Ordering of Operators

Because 2>&1 copies the current value of descriptor 1, the order of the operators changes the result. In "cmd > file 2>&1" standard output is redirected first, so the duplicate sends standard error into the file as well. In "cmd 2>&1 > file" the duplicate is made while descriptor 1 still refers to the terminal, so standard error stays on the terminal and only standard output goes into the file. For this reason the parser stores the redirections in an ordered array and applies them left to right rather than storing a single file name per stream.

```
cmd > file 2>&1 -> stdout: file stderr: file
cmd 2>&1 > file -> stdout: file stderr: terminal
```

5. Handling Edge Cases

The parser rejects an operator that is not followed by a file name, an operator immediately followed by another operator, an empty stage produced by two consecutive pipe symbols, and a redirection count beyond the array capacity. These are detected during parsing, before any descriptor is opened or any process is created.

6. Complex Commands: Pipes with Redirections

A complex command line mixes both features, for example "sort < in.txt | uniq -c | sort -rn > out.txt". The line is split on the pipe symbol into stages, and each stage is then parsed independently for its own redirections. The result is stored in a CommandLine structure holding an ordered array of Command structures.

```
typedef struct { enum RedirType type; char *file; } Redirection;
typedef struct { char *args[MAX_ARGS]; int argCount;
 Redirection redirs[MAX_REDIRS]; int redirCount; } Command; typedef struct
{ Command stages[MAX_STAGES]; int stageCount; } CommandLine;
```

7. Resolving the Execution Order

Inside each child process the pipe ends are connected first and the explicit redirections are applied afterwards. This ordering matters: if a stage carries its own redirection, that redirection must override the pipe rather than the other way round. The parent closes every pipe descriptor after forking, otherwise a reader would never see end-of-file and the pipeline would hang.

8. Validating Syntax of the Whole Line

Validation checks that at least one stage exists, that no stage is empty, that '<' appears only on the first stage because any later stage already receives its input from a pipe, and that '>' or '>>' appears only on the last stage because any earlier stage already sends its output into a pipe. A conflicting line is reported and never executed.

9. Generating an Execution Plan

Before execution the shell prints the exact sequence of operations it will perform for each stage: which pipe end is attached to which descriptor, which files are opened with which flags, which dup2() calls are issued, and finally the execvp() call. The plan makes the resolved execution order visible and is a convenient way to confirm that the ordering rules described above were applied correctly.

Algorithm:

Part 1 – Combined Redirection

• Start the program.
• Read the command line entered by the user.
• Tokenize the line on spaces and tabs.
• If a token is a redirection operator, record its type in the redirection array in the order written.
• If the operator needs a file, read the next token as the file name and report a syntax error if it is missing.
• Store every remaining token in the argument array.
• Fork a child process.
• In the child, process the redirection array from first to last.
• For a file operator, open the file and duplicate the descriptor onto 0, 1 or 2 as required. • For 2>&1, duplicate the current descriptor 1 onto descriptor 2 so the streams are merged. • Execute the command with execvp().
• Wait for the child, then reopen the target files and display their contents to verify the integrated output.
• Stop the program.

Part 2 – Complex Commands

• Start the program.
• Read the complete command line.
• Split the line on the pipe symbol to obtain each stage.
• Parse every stage into its argument array and its ordered redirection array.
• Validate that no stage is empty.
• Validate that '<' appears only on the first stage and '>' or '>>' only on the last stage.
• If invalid, report the conflict and skip execution.
• If valid, generate and display the execution plan for every stage.
• Create one pipe for each pair of consecutive stages.
• Fork one child process per stage.
• In each child, connect the pipe ends first and then apply the redirections in order.
• Execute each stage with execvp().
• Close all pipe descriptors in the parent and wait for every stage.
• Test valid scenarios and invalid ones such as an empty stage or a conflicting redirection. • Stop the program.

Functions Used:

Function	Purpose
main()	Starting point of the program
isRedirToken()	Recognises '<', '>', '>>', '2>', '2>>', '2>&1' and '1>&2' operators
redirName()	Returns the printable text of a redirection operator
parseCombinedRedirection()	Separates arguments from the ordered list of redirections in one stage

Function	Purpose
parseComplexCommand()	Splits the whole line on '|' and parses every stage into the

CommandLine structure

validateSyntax() Rejects empty stages and redirections that conflict with an attached pipe

generateExecutionPlan() Prints the exact sequence of pipe, open, dup2 and execvp steps for every stage

applyRedirections() Applies each redirection in written order, including the merging forms

executeCommandLine() Creates the pipes, forks one process per stage and executes the plan

verifyFileContents() Reads a file back to verify the merged or redirected data

createSampleInputFile() Creates the data file used by the test scenarios runScenario() Parses, validates, plans, executes and verifies a single test case testCombinedRedirection() Tests stream merging, separation and operator ordering testComplexCommands() Tests pipes combined with redirections and invalid scenarios open() Opens or creates a file and returns the lowest free file descriptor

dup2() Duplicates one descriptor onto another to redirect or merge a stream

close() Closes a descriptor once it has been duplicated or is unused pipe() Creates the read/write descriptor pair connecting two stages fork() Creates one child process per stage of the command line

execvp() Replaces the child image with the stage's command, keeping the descriptors

waitpid() Waits for every stage of the command line to finish

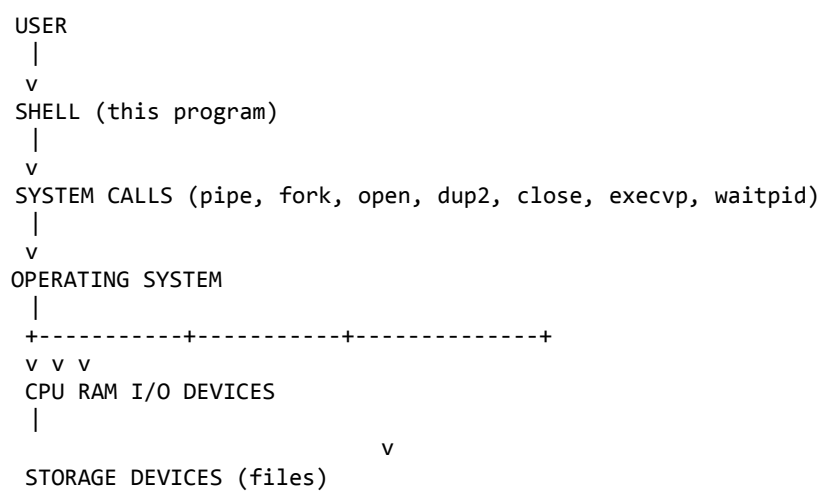
strtok_r() Reentrant tokenizer used for the outer '|' split and the inner token split

strerror() Converts errno into a readable error message

Hardware and Operating System Relationship

Hardware consists of physical components such as the CPU, RAM, storage devices, keyboard, mouse and display. The Operating System acts as an interface between application software and hardware. It manages processes, memory, files, devices and system resources.

Relationship



How OS Interacts with Hardware

- **Process Management** – The OS creates and schedules one process for every stage of the command line.
- **Memory Management** – The OS manages memory for the CommandLine structure, the pipe buffers and each process image.
- **File Management** – The OS maintains the descriptor table and the shared open file description used when descriptors are duplicated.
- **Device Management** – The OS switches each stream between the terminal device, a pipe and a storage device.
- **Security** – The OS enforces file permissions when the redirection targets are

opened. **Observation**

The program successfully parses a command line carrying several redirection operators, stores them in the order they were written, and applies them one after another inside the child process. When "> merged.txt 2>&1" is used, the descriptor duplication sends both the normal output and the error message of ls into the same file, and reading the file back confirms that the two streams were genuinely merged. Using separate targets keeps them apart, and the append form adds to the existing contents instead of replacing them.

The ordering test confirms that the position of 2>&1 changes the outcome: with "2>&1 > ordered.txt" the error message appears on the terminal while only the normal output reaches the file, because the duplicate was taken while descriptor 1 still referred to the terminal.

The complex-command module successfully parses lines containing both pipes and redirections, generates an execution plan listing every pipe connection, open, dup2 and execvp call, and executes the stages so that the first stage reads from its input file and the last stage writes into its output file. Invalid lines are rejected before execution, including an empty stage, a missing file name after an operator, an input redirection on a piped stage and an output redirection on a stage whose output already feeds a pipe.

Result

Combined redirection was successfully implemented, including parsing of multiple operators, duplication of file descriptors, merging of standard error into standard output, verification of operator ordering, handling of edge cases and testing of the integrated output written to disk. Complex command handling was also successfully implemented, including parsing of lines that mix pipes and redirections, resolution of the correct execution order, syntax validation of the whole line, generation of an explicit execution plan and testing across valid and invalid scenarios.

Design / Flow

Combined Redirection Flow

```
Read command line
 |
 v
Tokenize the stage
 |
 v
Token is an operator? --No--> append to args[]
 | Yes
 v
Record {type, file} into redirs[] IN WRITTEN
ORDER |
```

```
 v
fork() child
  |
 v
Apply redirs[0], redirs[1], ... one after another
 |
  +-----------------+-----------------+
  | | |
 v v v
 open + dup2(fd,1) open + dup2(fd,2) dup2(1,2) /* merge */
 |
  v
 execvp(command) --> streams already point at the right files
 |
  v
 Parent waitpid(), then verify the file contents
```

Complex Command Execution Flow

```
 Input: "cat < in.txt | grep x 2>&1 | wc -l >
out.txt" |
  v
 Split on '|' -> stage[0..n-1]
  |
  v
 Parse each stage: args[] + ordered redirs[]
  |
  v
Validate syntax
  | - stageCount > 0, no empty stage
  | - '<' only on the first stage
  | - '>' / '>>' only on the last stage
  |
  +---------+----------+
  Valid Invalid
  | |
  v v
Generate execution Report the conflict,
 plan (pipe/dup2/exec) skip execution
  |
  v
 Create (n-1) pipes, fork() each stage
  |
  v
 Child: connect pipe ends first, then apply
redirs[] |
  v
 execvp() each stage, parent waits for all
```

CODE-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/wait.h>
#define MAX_CMD_LEN 300
#define MAX_ARGS 20
#define MAX_STAGES 10
#define MAX_REDIRS 10
 /* ================= Redirection operation types ================= */
 enum RedirType {
  RED_IN, /* < : stdin from file */
  RED_OUT_TRUNC, /* > : stdout to file (overwrite) */
  RED_OUT_APPEND, /* >> : stdout to file (append) */
  RED_ERR_TRUNC, /* 2> : stderr to file (overwrite) */
```

```
 RED_ERR_APPEND, /* 2>> : stderr to file (append) */
 RED_ERR_TO_OUT, /* 2>&1 : duplicate stdout onto stderr */
 RED_OUT_TO_ERR /* 1>&2 : duplicate stderr onto stdout */
};

typedef struct {
 enum RedirType type;
 char *file; /* NULL for the duplicating forms */
} Redirection;

typedef struct {
 char *args[MAX_ARGS];
 int argCount;
 Redirection redirs[MAX_REDIRS]; /* kept in the order written */
 int redirCount;
} Command;

typedef struct {
 Command stages[MAX_STAGES];
 int stageCount;
} CommandLine;

/* ================= Part 1: Parsing combined redirection ================= */

static const char *redirName(enum RedirType t)
{
 switch (t) {
 case RED_IN: return "<";
 case RED_OUT_TRUNC: return ">";
 case RED_OUT_APPEND: return ">>";
 case RED_ERR_TRUNC: return "2>";
 case RED_ERR_APPEND: return "2>>";
 case RED_ERR_TO_OUT: return "2>&1";
 case RED_OUT_TO_ERR: return "1>&2";
 }
 return "?";
}

/* Returns 1 if the token is a redirection operator, and fills *type */
static int isRedirToken(const char *tok, enum RedirType *type)
{
 if (strcmp(tok, "<") == 0) { *type = RED_IN; return 1; }
 if (strcmp(tok, ">") == 0 || strcmp(tok, "1>") == 0) { *type = RED_OUT_TRUNC; return 1; } if
(strcmp(tok, ">>") == 0 || strcmp(tok, "1>>") == 0) { *type = RED_OUT_APPEND; return 1; } if
(strcmp(tok, "2>") == 0) { *type = RED_ERR_TRUNC; return 1; }
 if (strcmp(tok, "2>>") == 0) { *type = RED_ERR_APPEND; return 1; }
 if (strcmp(tok, "2>&1") == 0) { *type = RED_ERR_TO_OUT; return 1; }
 if (strcmp(tok, "1>&2") == 0) { *type = RED_OUT_TO_ERR; return 1; }
 return 0;
}

/* Parses one stage: separates arguments from the ordered redirection
list */ int parseCombinedRedirection(char *segment, Command *cmd)
{
 cmd->argCount = 0;
 cmd->redirCount = 0;

 char *saveptr;
 char *token = strtok_r(segment, " \t", &saveptr);
 while (token != NULL)
 {
 enum RedirType type;
 if (isRedirToken(token, &type))
 {
 if (cmd->redirCount >= MAX_REDIRS)
 {
 printf(" Syntax error: too many redirections in one stage.\n"); return 0;
 }
 cmd->redirs[cmd->redirCount].type = type;
 cmd->redirs[cmd->redirCount].file = NULL;
```

```
 if (type != RED_ERR_TO_OUT && type != RED_OUT_TO_ERR)
 {
 char *fileTok = strtok_r(NULL, " \t", &saveptr);
 enum RedirType dummy;
 if (fileTok == NULL || isRedirToken(fileTok, &dummy))
 {
 printf(" Syntax error: missing file name after '%s'.\n", redirName(type)); return
 0; }
 cmd->redirs[cmd->redirCount].file = fileTok;
 }
 cmd->redirCount++;
 }
 else if (cmd->argCount < MAX_ARGS - 1)
 {
 cmd->args[cmd->argCount++] = token;
 }
 token = strtok_r(NULL, " \t", &saveptr);
 }
 cmd->args[cmd->argCount] = NULL;
 return 1;
}

/* Splits the whole line on '|' and parses every stage */
int parseComplexCommand(char *input, CommandLine *cl)
{
 cl->stageCount = 0;

 char *saveOuter;
 char *segment = strtok_r(input, "|", &saveOuter);
 while (segment != NULL && cl->stageCount < MAX_STAGES)
 {
 if (!parseCombinedRedirection(segment, &cl->stages[cl->stageCount]))
return 0;
 cl->stageCount++;
 segment = strtok_r(NULL, "|", &saveOuter);
 }
 return 1;
}

/* ================= Part 2: Syntax validation ================= */
int validateSyntax(const CommandLine *cl)
{
 if (cl->stageCount == 0)
 {
 printf(" Invalid: no command found.\n");
 return 0;
 }
 for (int i = 0; i < cl->stageCount; i++)
 {
 const Command *c = &cl->stages[i];

 if (c->argCount == 0)
 {
 printf(" Invalid: stage %d is empty.\n", i + 1);
 return 0;
 }
 for (int j = 0; j < c->redirCount; j++)
 {
 enum RedirType t = c->redirs[j].type;

 /* input redirection is only meaningful on the first stage */
 if (t == RED_IN && i != 0)
 {
 printf(" Invalid: '<' on stage %d conflicts with the pipe feeding it.\n", i + 1); return 0;
 }
 /* stdout redirection is only meaningful on the last stage */
 if ((t == RED_OUT_TRUNC || t == RED_OUT_APPEND) && i != cl->stageCount - 1) { printf("
Invalid: '%s' on stage %d conflicts with the pipe leaving it.\n", redirName(t), i + 1);
 return 0;
 }
```

```
 }
 }
 printf(" Syntax valid: %d stage(s), %d pipe(s) required.\n",
 cl->stageCount, cl->stageCount - 1);
 return 1;
}

/* ================= Part 3: Execution plan generation
================= */ void generateExecutionPlan(const CommandLine *cl)
{
 printf(" Execution plan:\n");
 for (int i = 0; i < cl->stageCount; i++)
 {
 const Command *c = &cl->stages[i];
 printf(" Stage %d:", i + 1);
 for (int j = 0; j < c->argCount; j++)
 printf(" %s", c->args[j]);
 printf("\n");

 if (i != 0)
 printf(" step: fd 0 <- read end of pipe %d\n", i);
 if (i != cl->stageCount - 1)
 printf(" step: fd 1 -> write end of pipe %d\n", i + 1);

 /* the redirections are applied strictly left to right */
 for (int j = 0; j < c->redirCount; j++)
 {
 const Redirection *r = &c->redirs[j];
 switch (r->type)
 {
 case RED_IN:
 printf(" step: open(\"%s\", O_RDONLY), dup2(fd, 0)\n", r->file); break; case RED_OUT_TRUNC:
 printf(" step: open(\"%s\", O_WRONLY|O_CREAT|O_TRUNC), dup2(fd, 1)\n", r- >file); break;
 case RED_OUT_APPEND:
 printf(" step: open(\"%s\", O_WRONLY|O_CREAT|O_APPEND), dup2(fd, 1)\n", r->file); break;
 case RED_ERR_TRUNC:
 printf(" step: open(\"%s\", O_WRONLY|O_CREAT|O_TRUNC), dup2(fd, 2)\n", r- >file);
 break; case RED_ERR_APPEND:
 printf(" step: open(\"%s\", O_WRONLY|O_CREAT|O_APPEND), dup2(fd, 2)\n", r->file); break;
 case RED_ERR_TO_OUT:
 printf(" step: dup2(1, 2) /* stderr merged into current stdout */\n"); break;
 case RED_OUT_TO_ERR:
 printf(" step: dup2(2, 1) /* stdout merged into current stderr */\n"); break;
 }
 }
 printf(" step: execvp(\"%s\", args)\n", c->args[0]);
 }
}

/* ================= Part 4: Applying redirections in order
================= */ int applyRedirections(const Command *c)
{
 for (int j = 0; j < c->redirCount; j++)
 {
 const Redirection *r = &c->redirs[j];
 int fd;

 switch (r->type)
 {
 case RED_IN:
 fd = open(r->file, O_RDONLY);
 if (fd < 0) { fprintf(stderr, "cannot open '%s' (%s)\n", r->file,
strerror(errno)); return 0; }
 dup2(fd, STDIN_FILENO); close(fd);
 break;
 case RED_OUT_TRUNC:
 case RED_OUT_APPEND:
 fd = open(r->file, O_WRONLY | O_CREAT |
  (r->type == RED_OUT_APPEND ? O_APPEND : O_TRUNC), 0644); if (fd < 0) {
fprintf(stderr, "cannot open '%s' (%s)\n", r->file, strerror(errno)); return 0; }
dup2(fd, STDOUT_FILENO); close(fd);
```

```
 break;

 case RED_ERR_TRUNC:
 case RED_ERR_APPEND:
 fd = open(r->file, O_WRONLY | O_CREAT |
  (r->type == RED_ERR_APPEND ? O_APPEND : O_TRUNC), 0644); if (fd < 0) {
fprintf(stderr, "cannot open '%s' (%s)\n", r->file, strerror(errno)); return 0; }
dup2(fd, STDERR_FILENO); close(fd);
 break;

 case RED_ERR_TO_OUT:
 dup2(STDOUT_FILENO, STDERR_FILENO); /* duplicate descriptor 1 onto 2 */ break;

 case RED_OUT_TO_ERR:
 dup2(STDERR_FILENO, STDOUT_FILENO); /* duplicate descriptor 2 onto 1 */ break;
 }
 }
 return 1;
}

/* ================ Part 5: Execution ================ */
void executeCommandLine(CommandLine *cl)
{
 int numPipes = cl->stageCount - 1;
 int pipefds[2 * MAX_STAGES];
 for (int i = 0; i < numPipes; i++)
 if (pipe(pipefds + i * 2) < 0) { perror("pipe"); return; }

 pid_t pids[MAX_STAGES];

 for (int i = 0; i < cl->stageCount; i++)
 {
 pids[i] = fork();
 if (pids[i] < 0) { perror("fork"); return; }

 if (pids[i] == 0)
 {
 /* pipe connections first, redirections afterwards so that an explicit
'>' or '<' overrides the pipe on that stage */
 if (i != 0)
 dup2(pipefds[(i - 1) * 2], STDIN_FILENO);
 if (i != cl->stageCount - 1)
 dup2(pipefds[i * 2 + 1], STDOUT_FILENO);

 for (int j = 0; j < 2 * numPipes; j++)
 close(pipefds[j]);

 if (!applyRedirections(&cl->stages[i]))
 _exit(1);

 execvp(cl->stages[i].args[0], cl->stages[i].args);
 fprintf(stderr, "%s: command not found\n", cl->stages[i].args[0]);
_exit(127);
 }
 }

 for (int j = 0; j < 2 * numPipes; j++)
 close(pipefds[j]);
 for (int i = 0; i < cl->stageCount; i++)
 waitpid(pids[i], NULL, 0);
}

/* Reads a file back so merged/redirected output can be verified */
void verifyFileContents(const char *fileName)
{
 FILE *fp = fopen(fileName, "r");
     if (fp == NULL) { printf(" Cannot verify '%s' (%s)\n", fileName, strerror(errno)); return; }

 char line[MAX_CMD_LEN];
```

```
 printf(" --- contents of %s ---\n", fileName);
 while (fgets(line, sizeof(line), fp) != NULL)
 printf(" %s", line);
 fclose(fp);
}

void createSampleInputFile(const char *fileName)
{
 FILE *fp = fopen(fileName, "w");
 if (fp == NULL) { perror("fopen"); return; }
 fprintf(fp, "delta\napple\ncherry\nbanana\napple\ncherry\napple\n");
 fclose(fp);
 printf("Sample input file '%s' created with 7 lines.\n", fileName);
}

/* ================= Part 6: Test drivers ================= */
void runScenario(const char *input, const char *verifyFile)
{
 char buffer[MAX_CMD_LEN];
 strncpy(buffer, input, MAX_CMD_LEN - 1);
 buffer[MAX_CMD_LEN - 1] = '\0';
 printf("\nInput: \"%s\"\n", input);

 CommandLine cl;
 if (!parseComplexCommand(buffer, &cl))
 {
 printf(" Skipping execution: parsing failed.\n");
 return;
 }
 if (!validateSyntax(&cl))
 {
 printf(" Skipping execution due to invalid syntax.\n");
 return;
 }
 generateExecutionPlan(&cl);
 printf(" Output:\n");
 executeCommandLine(&cl);
 if (verifyFile != NULL)
 verifyFileContents(verifyFile);
}

void testCombinedRedirection(void)
{
 printf("\n=== Part 1: Testing Combined Redirection and Stream Merging ===\n");
createSampleInputFile("input.txt");

 /* 'ls' writes the existing name to stdout and the error to stderr,
which makes the merging of the two streams visible */
 runScenario("ls input.txt missing.txt > merged.txt 2>&1", "merged.txt"); runScenario("ls
input.txt missing.txt > only_out.txt 2> only_err.txt", "only_out.txt");
verifyFileContents("only_err.txt");
 runScenario("ls input.txt missing.txt 2>&1 > ordered.txt", "ordered.txt");
runScenario("echo appended line >> merged.txt 2>&1", "merged.txt"); }

void testComplexCommands(void)
{
 printf("\n=== Part 2: Testing Complex Commands (Pipes + Redirections) ===\n");

 runScenario("sort < input.txt | uniq -c | sort -rn > result.txt", "result.txt");
runScenario("cat < input.txt | grep apple | wc -l", NULL);
 runScenario("ls input.txt missing.txt 2>&1 | wc -l", NULL);

 printf("\n--- Invalid scenarios (must be rejected before execution) ---\n");
runScenario("ls | | wc -l", NULL);
 runScenario("sort < input.txt >", NULL);
    runScenario("cat input.txt | sort < input.txt", NULL);
    runScenario("ls > out.txt | wc -l", NULL);
    }

    /* ================= Main ================= */
```

```
int main(void)
{
setvbuf(stdout, NULL, _IONBF, 0); /* keep parent and child output in order */
testCombinedRedirection();
testComplexCommands();
return 0;
}
```

Compilation and Execution

1. Compile

```
gcc question15.c -o
```

```
question15
```

2. Execute

```
./question15
```

Sample Output

```
Output:
--- contents of merged.txt ---
ls: cannot access 'missing.txt': No such file or directory
input.txt
appended line

=== Part 2: Testing Complex Commands (Pipes + Redirections) ===

Input: "sort < input.txt | uniq -c | sort -rn > result.txt"
Syntax valid: 3 stage(s), 2 pipe(s) required.
Execution plan:
  Stage 1: sort
    step: fd 1 -> write end of pipe 1
    step: open("input.txt", O_RDONLY), dup2(fd, 0)
    step: execvp("sort", args)
  Stage 2: uniq -c
    step: fd 0 <- read end of pipe 1
    step: fd 1 -> write end of pipe 2
    step: execvp("uniq", args)
  Stage 3: sort -rn
    step: fd 0 <- read end of pipe 2
    step: open("result.txt", O_WRONLY|O_CREAT|O_TRUNC), dup2(fd, 1)
    step: execvp("sort", args)
Output:
--- contents of result.txt ---
      3 apple
      2 cherry
      1 delta
      1 banana

Input: "cat < input.txt | grep apple | wc -l"
Syntax valid: 3 stage(s), 2 pipe(s) required.
Execution plan:
  Stage 1: cat
    step: fd 1 -> write end of pipe 1
    step: open("input.txt", O_RDONLY), dup2(fd, 0)
    step: execvp("cat", args)
  Stage 2: grep apple
    step: fd 0 <- read end of pipe 1
    step: fd 1 -> write end of pipe 2
    step: execvp("grep", args)
  Stage 3: wc -l
    step: fd 0 <- read end of pipe 2
    step: execvp("wc", args)
Output:
3
```

```
=== Part 1: Testing Combined Redirection and Stream Merging ===
Sample input file 'input.txt' created with 7 lines.

Input: "ls input.txt missing.txt > merged.txt 2>&1"
Syntax valid: 1 stage(s), 0 pipe(s) required.
Execution plan:
  Stage 1: ls input.txt missing.txt
    step: open("merged.txt", O_WRONLY|O_CREAT|O_TRUNC), dup2(fd, 1)
    step: dup2(1, 2) /* stderr -> stdout */
    step: execvp("ls", args)
Output:
--- contents of merged.txt ---
ls: cannot access 'missing.txt': No such file or directory
input.txt

Input: "ls input.txt missing.txt > only_out.txt 2> only_err.txt"
Syntax valid: 1 stage(s), 0 pipe(s) required.
Execution plan:
  Stage 1: ls input.txt missing.txt
    step: open("only_out.txt", O_WRONLY|O_CREAT|O_TRUNC), dup2(fd, 1)
    step: open("only_err.txt", O_WRONLY|O_CREAT|O_TRUNC), dup2(fd, 2)
    step: execvp("ls", args)
Output:
--- contents of only_out.txt ---
input.txt
--- contents of only_err.txt ---
ls: cannot access 'missing.txt': No such file or directory

Input: "ls input.txt missing.txt 2>&1 > ordered.txt"
Syntax valid: 1 stage(s), 0 pipe(s) required.
Execution plan:
  Stage 1: ls input.txt missing.txt
    step: dup2(1, 2) /* stderr -> stdout */
    step: open("ordered.txt", O_WRONLY|O_CREAT|O_TRUNC), dup2(fd, 1)
    step: execvp("ls", args)
Output:
ls: cannot access 'missing.txt': No such file or directory
--- contents of ordered.txt ---
input.txt

Input: "echo appended line >> merged.txt 2>&1"
Syntax valid: 1 stage(s), 0 pipe(s) required.
Execution plan:
  Stage 1: echo appended line
    step: open("merged.txt", O_WRONLY|O_CREAT|O_APPEND), dup2(fd, 1)
    step: dup2(1, 2) /* stderr -> stdout */
    step: execvp("echo", args)
Output:
--- contents of merged.txt ---
```

```
Input: "ls input.txt missing.txt 2>&1 | wc -l"
Syntax valid: 2 stage(s), 1 pipe(s) required.
Execution plan:
  Stage 1: ls input.txt missing.txt
    step: fd 1 -> write end of pipe 1
    step: dup2(1, 2) /* stderr -> stdout */
    step: execvp("ls", args)
  Stage 2: wc -l
    step: fd 0 <- read end of pipe 1
    step: execvp("wc", args)
Output:
2

--- Invalid scenarios (must be rejected before execution) ---

Input: "ls | | wc -l"
Invalid: stage 2 is empty.
Skipping execution: parsing failed.

Input: "sort < input.txt >"
Syntax error: missing file name after '>'.
Skipping execution: parsing failed.

Input: "cat input.txt | sort < input.txt"
Invalid: '<' on stage 2 conflicts with the pipe feeding it.
Skipping execution due to invalid syntax.

Input: "ls > out.txt | wc -l"
Invalid: '>' on stage 1 conflicts with the pipe leaving it.
Skipping execution due to invalid syntax.
```

```
=== Part 1: Testing Command History ===
Adding command: pwd
Adding command: ls -l
Adding command: cd /tmp
Adding command: echo hello
Adding command: whoami
Adding command: date
Adding command: uname -a

--- Command History (most recent last) ---
 3: cd /tmp
 4: echo hello
 5: whoami
 6: date
 7: uname -a

--- Validating History Consistency ---
OK: historySize (5) matches expected buffer occupancy (5)
Total commands ever added: 7 | Currently retained: 5 (capacity 5)

Retrieving entry #2 from current buffer: echo hello
Retrieving out-of-range entry #99: (not found)

=== Part 2: Testing Pipeline Configurations ===

Input: "ls"
  Stage 1: ls
Pipeline valid: 1 stage(s), 0 pipe(s) needed.
Output:
Experiment-2       ordered.txt      question9
filecopy.c.save    os_task          question9.c
filecopy.c.save.1  os_task.c        quotes
fork               output.txt       quotes.c
fork.c             parser           readonly.txt
hello              parser.c         redirection
hello.c            path_resolver    redirection.c
input.txt          path_resolver.c  redirection_pipeline
linux              question         redirection_pipeline.c
merged.txt         question.c       result.txt
only_err.txt       question7
only_out.txt       question7.c

Input: "ls | wc -l"
  Stage 1: ls
  Stage 2: wc -l
```

```
Input: "ls | wc -l"
  Stage 1: ls
  Stage 2: wc -l
Pipeline valid: 2 stage(s), 1 pipe(s) needed.
Output:
34

Input: "ls | grep c | wc -l"
  Stage 1: ls
  Stage 2: grep c
  Stage 3: wc -l
Pipeline valid: 3 stage(s), 2 pipe(s) needed.
Output:
15

Input: "ls | | wc"
  Stage 1: ls
  Stage 2:
  Stage 3: wc
Pipeline invalid: stage 2 is empty.
Skipping execution due to invalid pipeline layout.
```

Conclusion

This experiment demonstrates combined redirection in a custom shell, including parsing of several operators on one command line, duplication of file descriptors with dup2(), merging of standard error into standard output through the 2>&1 form, and verification that the operators take effect strictly in the order they are written. It also demonstrates the handling of complex command lines that mix pipes with redirections, including syntax validation that rejects conflicting operators before execution and the generation of an explicit execution plan describing every system call. Together, these illustrate important Operating System concepts such as the process descriptor table, shared open file

descriptions, inter-process communication through pipes, and the separation between a program and the streams the kernel attaches to it.